ASSIGNMENT 1

PAYMENT CALCULATOR FOR A VILLAGE MILK CO-OPERATIVE

TASK 1: Understand the Problem and Identify the Data

Bloom's Level: K4 – Analyse
Marks: 4

1.1 Problem Understanding

The Ksheera Village Milk Producers' Co-operative collects milk from approximately 300 farmers twice a day. The payment calculator must calculate the correct price per litre based on the milk's fat and solids-not-fat (SNF) percentages. It must calculate each farmer's payment and the total payment for a ten-day period.

The calculator should avoid incorrect totals, accept valid entries and store rate values in one place so that future rate revisions can be applied easily.

1.2 Inputs and Outputs

Inputs

Input	Data type	Purpose
Farmer ID	String (str)	Uniquely identifies the farmer; it may contain letters or numbers.
Milk quantity in litres	Decimal number	Records the quantity of milk supplied.
Fat percentage	Decimal number	Determines the fat-based rate adjustment.

SNF percentage	Decimal number	Determines the SNF-based rate adjustment.
Shift	String (str)	Identifies whether milk was supplied during the morning or evening shift.

Outputs

Output	Data type	Purpose
Rate per litre	Decimal number	Shows the calculated price for one litre of milk.
Payment amount	Decimal number	Shows the amount payable for one milk entry.
Ten-day total	Decimal number	Shows the total payment for a farmer across the ten-day payment period.

Decimal arithmetic is suitable for monetary calculations because the system must produce accurate payments to the nearest paisa.

1.3 Named Constants

The following values are defined as constants because they are fixed for a particular rate-card version.

Constant	Value	Meaning
BASE_RATE	₹30.00	Base price per litre.
BASE_FAT	3.5%	Reference fat percentage.
BASE_SNF	8.5%	Reference SNF percentage.
FAT_STEP	0.1%	Fat percentage change for one rate adjustment step.
FAT_ADJUSTMENT	₹0.40	Rate change for each fat step.
SNF_STEP	0.1%	SNF percentage change for one rate adjustment step.
SNF_ADJUSTMENT	₹0.30	Rate change for each SNF step.
MIN_FAT	2.0%	Minimum accepted fat percentage.
MAX_FAT	7.0%	Maximum accepted fat percentage.

- Named constants make the program easier to maintain. If the co-operative changes its rates, the programmer can update the values in one place instead of searching through multiple expressions.
- These values are fixed for the current calculation, but future rate revisions may require updating the rate card.

1.4 Whole Numbers, Fractional Numbers and Text

- **Whole-number values:** A count of entries or the number of farmers can be represented using `int`. A farmer ID should not automatically be converted to an integer because it may contain letters or leading zeros.
- **Fractional values:** Milk quantity, fat percentage, SNF percentage and rate per litre may contain fractional values. Decimal arithmetic should be used where exact decimal representation is important, especially for payments.
- **Text values:** Farmer ID and shift can be represented using `str`. For example, a shift may be recorded as `"Morning"` or `"Evening"`.
- **Boolean values:** Validation checks may produce `True` or `False`, indicating whether an entry satisfies a rule.

1.5 Rules for Correct Payment

- Milk quantity must not be negative. An actual milk delivery should normally contain a positive quantity.
- Fat percentage must be between 2.0% and 7.0%, inclusive.
- SNF percentage must be numeric and within the valid range configured by the co-operative. The assignment does not specify its minimum and maximum limits, so these must be confirmed rather than invented.
- Farmer ID must be present and associated with the correct farmer.
- Shift must be a recognised value, such as `"Morning"` or `"Evening"`.
- The rate must increase or decrease according to the specified fat and SNF adjustments.
- The payment amount must be calculated using the quantity and rate per litre.
- The ten-day total must include all valid payments belonging to the same farmer during that period.
- Monetary calculations must avoid floating-point accumulation errors and must follow a consistent rounding policy to the nearest paisa.

1.6 Assumptions

- Each farmer has a unique identifier.
- Each valid milk entry belongs to a particular farmer, date and shift.
- The supplied base rate and adjustment values remain unchanged during a calculation unless an authorised rate revision is introduced.
- The ten-day total is calculated by adding the payments for all valid entries belonging to a farmer during that period.
- The co-operative will provide the valid SNF range and confirm the required rounding policy.
- If the automatic milk analyser is introduced, its readings will be validated before payment calculation.
- These assumptions make the system requirements clearer and help prevent incorrect or incomplete payments.

1.7 Alternative Concept and Complexity Analysis

Alternative 1: Separate variables for each farmer

For 300 farmers, the program could use separate variables such as:

```
litres1 = 10.5
fat1 = 4.0
snf1 = 8.7

litres2 = 8.0
fat2 = 3.8
snf2 = 8.5
```

The same pattern would need to be repeated for every farmer. This makes the program long, difficult to maintain and unsuitable for large numbers of farmers. Adding a new farmer would require creating more variables and potentially adding more calculation code.

Alternative 2: Store each entry in a list or tuple

A better approach is to store the values of each entry together in a structured record.

```
entries = [
   ("F001", 10.5, 4.0, 8.7, "Morning"),
   ("F002", 8.0, 3.8, 8.5, "Evening")
]
```

Each record contains the farmer ID, milk quantity, fat percentage, SNF percentage and shift. The program can process every entry using a loop.

Complexity comparison

Factor	Separate variables	List or tuple records
Processing	Repeated, manually written calculations	A loop processes each entry.
Time complexity	Difficult to maintain and scale; a loop-based version would take O(n) time	O(n) time to process n entries.
Storage	O(n) values are needed if all entries are stored	O(n) storage for n stored records.
Adding farmers	Requires additional variables and code	Add another record to the collection.
Maintenance	Difficult when the number of farmers grows	Easier to update and reuse.

Both designs require storage for the data if every entry is retained. However, a list of records makes processing, validation and maintenance much easier. If records are processed one at a time without storing the entire collection, additional working space can be O(1), excluding the input and output storage.

1.8 Conclusion

The payment calculator needs clearly defined inputs, outputs, constants, data types and validation rules. A list of structured records is more maintainable than separate variables for every farmer. Using Decimal arithmetic for monetary values and keeping rate values in named constants will help the co-operative produce accurate payments and accommodate future growth.

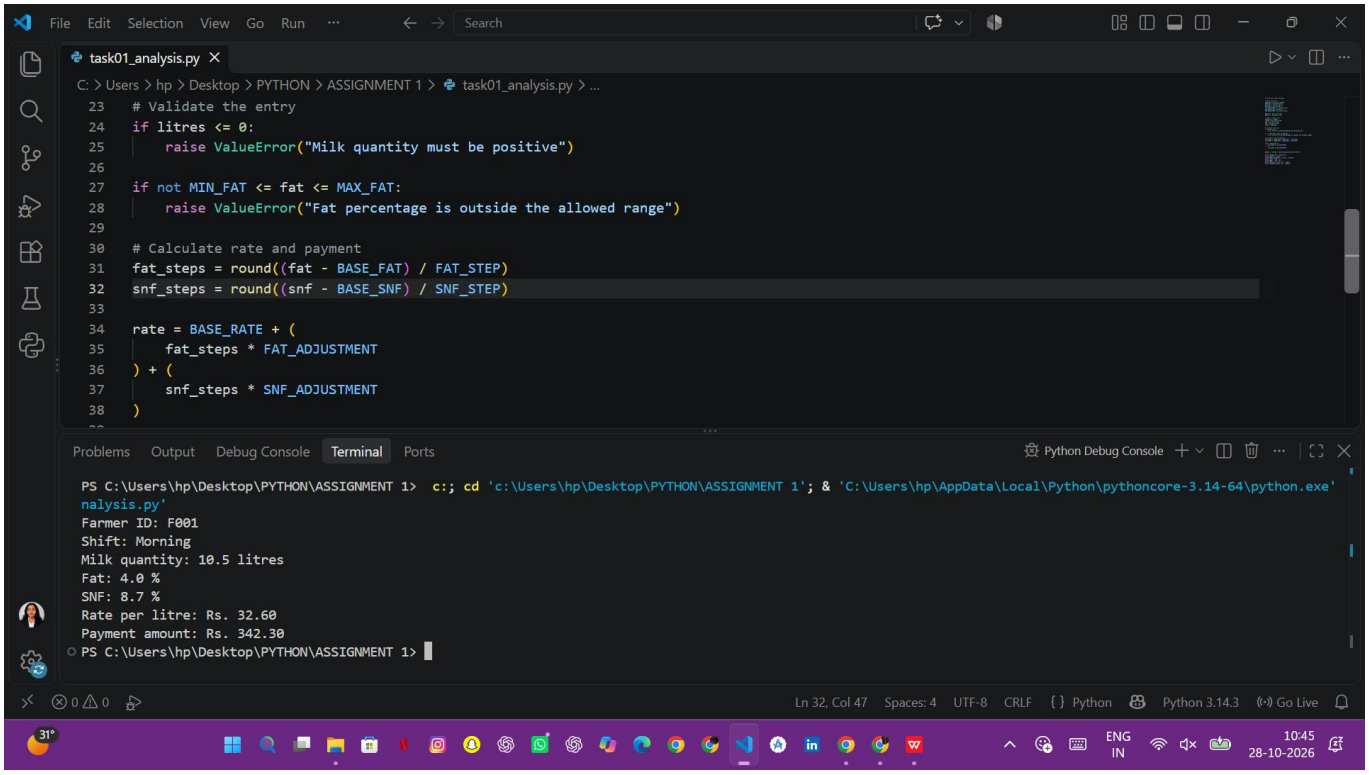


Figure 1: Task 1 Python execution output.